

P2005R0

2D Graphics: A Brief Review

Published Proposal, 2019-12-21

This version:

[none](#)

Author:

[James Berrow](#)

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Table of Contents

1	Acknowledgments
2	Abstract
3	Background Information
3.1	Colours
3.2	Origins
3.3	GPU Architecture
3.4	Batching
3.5	Document Scope
4	Colour Management
4.1	rgba_color
4.2	Linear Colour vs. sRGB in the API
5	Vocabulary Types
5.1	Already Widely Misused
5.2	Long Hidden Issues In Plain Sight
5.2.1	Technical Defects

- 5.2.2 Problematic Design
- 5.2.3 Reference Implementation Defects

6 The Wider Graphics API

- 6.1 2D Graphics APIs in Practice
 - 6.1.1 Hardware Acceleration
- 6.2 Separation of CPU and GPU
- 6.3 Performance Limitations
- 6.4 Functionality Limitations

7 Performance, Safety, and API Usability

- 7.1 Fully Automatic Batching
- 7.2 Performance Issues Lead To Unsafety
- 7.3 Unsafety Leads To Usability Limitations
- 7.4 Potentially Unsolvable

8 Bad For Beginners

9 Conclusion and Recommendations

10 Misc. Errors

- 10.1 Linear Colour
- 10.2 Others

11 Footnotes

- 11.1 Async
- 11.2 Dear ImGui
- 11.3 operator*
- 11.4 Hidden API Cost

§ 1. Acknowledgments

Thanks to STL for encouraging me to write this paper, helping to filter ideas, providing detailed review, as well as navigating me through the C++ paper submission process.

§ 2. Abstract

This document aims to review portions of the proposed 2D graphics library "[P0267R10](#)", and focuses particularly on its handling of colours, linear algebra, the high level API design, and concerns around performance. There are a number of major API design issues and minor technical faults present in P0267 currently, which hamper current and future functionality, and more concerningly it may be of limited value to beginners. This document is aimed at people who do not have any background in computer graphics but want to understand the issues in depth, and additionally presents a different starting point for jumping off with a future graphics initiative.

§ 3. Background Information

§ 3.1. Colours

There are two relevant methods for how you might store a specific colour - [linear colour](#), and [sRGB](#). The short version is - linear colour is suitable for doing maths on, and sRGB is not. Linear colour is not something you put on a display, whereas sRGB is. Mixing up which kind of colour you are using is a very common error in graphics - many libraries fail to handle this correctly, and this leads to incorrect output. Even further, due to the prevalence of incorrectness, some technologies like CSS [deliberately](#) handle linear colour incorrectly.

Most colour data that users will plug into a program (e.g. searching for colour values) is in sRGB - linear colour is not generally used as a human consumable format. Importantly, 8 bit linear colour cannot represent every colour in 8 bit sRGB, so this won't change anytime soon. Generally any unspecified triplet of integer values that a user inputs is sRGB.

It is important to note that the meaningful distinction between linear colour and sRGB is not an optional concept. Not handling this isn't lacking a feature or a minor bug, it is incorrect in a major way.

§ 3.2. Origins

The API is largely based on Cairo, and a reference implementation for this proposed library is available over here https://github.com/cpp-io2d/P0267_RefImp.

§ 3.3. GPU Architecture

GPUs operate largely in a pipelined fashion. This is essentially a queue of commands - keeping this queue not empty is essential for performance, as is using the queue asynchronously. Blocking, synchronous commands are very bad for performance, in particular reads from the GPU to the CPU. While GPUs are massively parallel devices with conceptually thousands of threads, the submission of commands to a GPU is [largely a single threaded affair](#). There are specific use cases where multithreading is useful to performance, as in the DX12/Vulkan/Mantle/Metal APIs, but this is out of scope for both this paper, and the design goals of the 2D graphics proposal in general.

§ 3.4. Batching

There is a non trivial amount of overhead in issuing any GPU command. For getting the best performance, multiple commands can be merged together into one larger command. This isn't something that can be done unconditionally - the ability to 'batch' commands together into a single command is determined by certain constraints (primarily: sharing render state, like textures).

SFML/SDL/Allegro/Cairo and most other libraries do not *fully* automatically batch commands together - either intentionally, or due to difficulty implementing a fully automatic batching system and [API problems](#) (e.g. Skia). [Dear ImGui](#) is an example of a library that is largely able to fully automatically batch for performance, due to its specific use case for rendering GUIs and its API design. [SDL batches internally](#) where it is able, but relies on the programmer structuring their data in such a way that it can take advantage of this (e.g. constructing a sprite atlas).

To clarify terminology: I am using batching to refer to any scheme by which multiple independent GPU commands might be merged together (e.g. vertex lists in SFML, which are not strictly batching under some definitions, but function similarly and fulfill a similar use case), automatic batching to refer to the implementation making relatively simple decisions about which content to merge (e.g. as in SDL), and fully automatic batching to refer to an implementation which is able to make enough decisions about merging content together that manual batching is unnecessary in the general case, without having to make large code modifications to make it work. P0267 is proposing a fully automatically batched scheme under this terminology, as manual batching is not a high priority and the authors expect good performance without any manual batching.

§ 3.5. Document Scope

I have only reviewed things about which I have experience with, with respect to P0267, which are: Colour management, graphics APIs, linear algebra, font rendering, and performance.

§ 4. Colour Management

§ 4.1. `rgba_color`

The class "`rgba_color`" is the primary class involved in handling colours in P0267. This class is specified to be in linear colour (here called the RGBA color model). "Color models are often termed linear while color spaces are often termed gamma corrected" defines colour models as being linear, and colour spaces as being non linear, with this class going on to be defined as "using the RGBA color model".

The class has two constructors, one which takes floating point values between 0 and 1, and another which takes integral values between 0 and 255, both in linear colour. This might seem reasonable at first glance, with the caveat that integer linear colour is very rarely what you want.

The real problem comes in that most developers, especially beginners, have no idea about the difference between linear colour and sRGB. Partly due to 8-bit per channel integer linear colour not being a very useful format (and partly due to convention), the internet and other sources use the sRGB colour space to specify commonly used colours - e.g. "antique white" is [250, 235, 215](#), defined in the sRGB colour space.

The authors of P0267 fall into this exact trap by using the integer constructor to input sRGB constants - so as specified, all the colour constants in the paper are incorrect if `rgba_color` is truly intended to be linear colour. This is exacerbated by a lack of strong typing around colours, and a lack of API design around colour spaces at all. There is no built in support for explicitly disambiguating between sRGB and linear colour.

That said, it is not entirely clear how `rgba_color` is intended to be used. All practical usages of the class seem to imply that it stores sRGB data, including the text of P0267 itself, as well as the reference implementation which uses a non linear RGB colour space in the implementation when handling colours. However, the class provides an operator* which [cannot be used on sRGB data correctly](#), and gradients specify linear interpolation without colour space conversions which is incorrect for sRGB data, with the reference implementation making further mistakes.

These are all very common errors that are made when it comes to linear colour vs. sRGB (see [§ 10 Misc. Errors](#) for further mistakes).

§ 4.2. Linear Colour vs. sRGB in the API

To follow on from the above example, a pertinent question to then ask is: why does the computer graphics industry use linear colour at all? What is the purpose if it adds complexity without actually

doing anything - couldn't you just convert all your sRGB constants to linear colour under the hood when we get to the actual rendering bit (a potentially valid approach) and use sRGB everywhere?

The sole purpose of linear colour generally is that doing mathematical operations on the colours does the correct thing. Averaging two linear colours is perfectly fine, whereas averaging two sRGB colours will give an incorrect result - sRGB should be treated as a black box for all intents and purposes. It is therefore particularly desirable to use linear colour in API types, because it guarantees that any mathematical operations done by users will be correct (e.g. linear interpolation, a very common operation on colours that is commonly performed incorrectly).

Using linear colour as the exposed API type is definitely a sensible choice overall, with the caveat that humans don't tend to consume linear colours - which necessitates that appropriate (currently lacking) safeguards should be put in place to make it hard for users to input sRGB triples where linear colours are expected. Defining the use of linear colour everywhere means that you always know what your colour space is, and you can always perform correct colour operations without expensive colour space conversions. It would also be ideal if `rgba_color` could provide common vector operations, as fundamentally they are correct and useful for the type as specified.

While so far this might seem simply like P0267 is missing a few things then and that it could be easily rectified with extensions and fixes, this is where the real problems set in. The above issues are fixable, but the below issues become increasingly problematic.

§ 5. Vocabulary Types

As it stands, the class `"rgba_color"` has some problems with it. Outside of the proposed 2D graphics library, linear RGB colour is in (good) practice a 3-4 component proper vector type, with a rich set of functions and a full set of operators, parameterised, and preferably strongly typed with respect to colour spaces (linear colour vs. sRGB at minimum, and potentially HSL if supported etc.).

The other vector type present in the library is `basic_point_2d`, which in an ideal world would also be a fully featured vector class. Its functionality *should* largely overlap with that of `rgba_color` - most mathematical operations should be expressible on both of them, with the benefit that type safety could restrict some operations to only the types that benefit from them.

Unfortunately, the proposed 2D graphics library does not propose a vector type with which to base `basic_point_2d` and `rgba_color` on. Making the changes necessary to these classes to base them on proper vector types and provide useful vocabulary types post-hoc would result in breaking changes, which means that once these are standardised in their current temporary form, they'll persist while being insufficient. In their current form additionally, almost all specified operations should be duplicated between them as both are suitable for any provided vector functionality.

It is currently not possible to write generic templated code that accepts both `basic_point_2d` and `rgba_color` without duplication, for no reason. When a future more complete vector type appears (e.g. ["P1385R3"](#)) which is necessarily very different due to parameterisation, we then may well end up with 3 different incompatible vector APIs in the C++ standard that users have to write code for if they want their code to be flexible, rather than the expected number which is 1. The class `basic_matrix_2d` also suffers from genericity issues, with hardcoded accessor properties and no generic API, which results in similar problems with future matrix types.

`rgba_color` `operator*` is also very confusing. It saturates, meaning that it does not store values > 1 (instead clamping), but also does not allow multiplication by any floating point values > 1 or < 0 , or integer values > 255 or < 0 . Additionally, it overloads on float and int to define int multiplication as `/255.f`, the combined sum of which is best illustrated in the following code sample.

```
rgba_color some_col(0, 0.3, 0.5, 1); // illustrative example

rgba_color col1 = some_col * 255; // gives 0, 0.3, 0.5, 1
//rgba_color col2 = some_col * 255.; // not allowed

rgba_color col3 = some_col * 2; // gives 0, 0.0024, 0.0039, 0.0078
//rgba_color col4 = some_col * 2.; // not allowed

rgba_color col5 = (some_col * 2) * 0.5; // gives 0, 0.0012, 0.002, 0.0039
//rgba_color col6 = (some_col * 2.) * 0.5; // not allowed

//rgba_color col7 = some_col * (4 * 0.5); // not allowed
rgba_color col8 = (some_col * 4) * 0.5; // gives 0, 0.0024, 0.0039, 0.0078
```

While not explicitly wrong, this is very unintuitive behaviour, and will definitely be used incorrectly by beginners. It encourages users to mix integer and floating point types to get correct behaviour, and breaks commutativity.

§ 5.1. Already Widely Misused

The current ad-hoc approach to designing these types and lack of library fundamentals is likely why `rgba_color` is so easy to misuse. Colour is near universally mishandled across many open source libraries, but it is particularly disappointing that these mistakes might be cemented into C++ solely as a result of insufficient API design. Users will likely treat `rgba_color` as a black box due to its lack of useful API, input sRGB data instead into a traditional vector class not designed for colour management, perform incorrect maths on it, and then incorrectly convert to `rgba_color` at the end

ignoring any potentially safer constructors that might be introduced - this is particularly true if a future independent incompatible vector library proposal is standardised, which does not provide colour support due to being a niche field. `operator*` also has a particularly odd design, and seems error prone.

While these issues might seem theoretical or improbable, there are already real world examples of how much trouble this class is causing. For an explicit case of how deficient the `rgba_color` class is in practice, the complexities of linear colour management, and the importance of strong typing: the reference implementation is entirely incorrect with how it handles colours, and produces wrong output in a wide variety of cases across 3 separate backends, all implemented by different people. Precise examples are provided at the end of this paper under [§ 10 Misc. Errors](#) to avoid sidetracking, but the above issues ring extremely true in currently available code.

The argument that any minimal type is necessary and sufficient for the proposed 2D graphics library to be able to move forward is therefore not compelling here - ad-hoc specification will make the life of future programmers more difficult due to lack of foresight on difficult design problems, resulting in a type like `rgba_color` which is actively harmful in its current form, which even further may be accidentally being designed to fulfill a use case that it is not correct or intended for (storing sRGB data). This type needs proper thought put into its APIs, to prevent exactly the problems which are already prevalent with `rgba_color` in practice. An acceptable `rgba_color` would necessarily look very different to the type currently specified, with a large rethink necessary to prevent these issues.

§ 5.2. Long Hidden Issues In Plain Sight

Overall, there are a number of issues identified surrounding the handling of colours that have hidden in plain sight, seemingly for multiple years. It is surprising and concerning that none of these have been spotted before now.

§ 5.2.1. Technical Defects

1. Incorrect colour constants
2. The definition of how to perform premultiplied alpha is incorrect (sRGB data must be linearised first)
3. `operator*` saturates, but `rgba_color`'s API requirements prevent it from having an operation performed on it that would require this
4. All pixel formats are specified as being in the RGB colour model (linear), but are all integer formats. Precision loss is guaranteed if implemented as per exact wording (as converting 8-bit linear to 8-bit sRGB for rendering is incorrect). Slightly different wording could change these formats to be sRGB, but then linear blending requires sRGB textures which are not always

available, so it is unclear how this could be standardised. There is no specified high precision storage type which is suitable for storing linear data currently in P0267.

§ 5.2.2. Problematic Design

1. Ambiguously colour spaced constructors. Having unnamed constructors is probably a bad idea overall.
2. Integer linear constructor is a beginner trap.
3. `operator*` has very peculiar overloaded behaviour on `int` and `float`, and saturates by API design (e.g. you may write `col * 0.9`, but not `col * 1.1`).
4. No strong typing or signposting for colour spaces to common avoid linear vs. sRGB errors.
5. No functions to convert from or to sRGB, which are necessary given the ubiquitousness of sRGB and the necessity of linear colour.
6. Lack of useful features encourages the use of less appropriate classes not explicitly designed for storing colour.
7. Lack of interoperability with `basic_point_2d`, and likely future vector types.

§ 5.2.3. Reference Implementation Defects

1. Blending is incorrectly performed in sRGB.
2. `rgba_color` is widely misused.
3. The internal implementation uses sRGB as its colour space.
4. Most of the test data is incorrect due to sRGB blending, and appears to be testing the implementation against its own output.
5. Premultiplied alpha is implemented incorrectly.

Proper linear colour handling is one of the few fields where I consider myself to be relatively technically competent due to my experience working on subpixel font rendering and game engines, and is the area of P0267 I was most expertly able to review. I would consider it likely that there are other issues in the proposed 2D graphics library that I am not able to find due to a lack of specialist technical expertise in other fields.

§ 6. The Wider Graphics API

The above sections focus on the linear algebra section of the proposed 2D graphics library, and from here on will now focus on the actual graphics API itself.

§ 6.1. 2D Graphics APIs in Practice

The main bulk of the comparisons here will be to equivalent libraries such as SDL and SFML, both very widely used 2D graphics libraries that have a long history of game development and interactive applications. SFML is explicitly aimed at beginners to graphics programming and is generally considered a high quality C++ library, whereas SDL is a C library with a longer history behind it, and is virtually the standard for cross platform graphics applications. Both are widely used, and provide largely everything that the goals of the 2D graphics proposal intended to meet. SFML does not provide a software renderer, whereas SDL provides both a software and hardware renderer. While SFML can be somewhat limited in functionality for experts (but still provides extremely adequate facilities), SDL in particular successfully services the entire industry from beginner to expert.

Cairo by comparison, the library that this proposed API is based on, is fundamentally designed as a software renderer, which means that much of its API is based on that assumption - hardware acceleration for Cairo is considered experimental, and it is important to understand it through this light. Many applications have moved away from Cairo due to [fundamental](#) performance limitations, and it is not used for game development at all, not even at an extreme beginner level. Cairo also does not provide any solution for input, unlike alternatives, which often (though not always, [e.g. GLFW](#) which is a lower level API) have a clean unifying mechanism for handling both input and some rendering notifications (e.g. resizing), through a very general [events](#) system.

§ 6.1.1. Hardware Acceleration

It is extremely important to note that while Cairo is 'hardware accelerated', this is a rather broad term that does not mean what most people might think - the reality is that only portions of the Cairo implementation are fully hardware accelerated with acceptable performance on some backends, in large due to the API constraints of Cairo itself. It is not clear that it is possible for Cairo or the proposed 2D graphics library to be fully hardware accelerated, and given the prior difficulties in accomplishing this in Cairo, as well as the additional factor of the industry at large moving away from this API due to precisely this problem, it should not be considered a given until it has been done.

The Cairo [OpenGL backend](#) is explicitly stated not to be high performance by Cairo itself, stating "the canvas model does not often translate efficiently to the GPU", and that "In order to gain the most performance, you need to construct your UI fully cognisant of the GPU and program within its

capabilities. That is highly device and driver specific". The OpenGL backend has been experimental since its introduction, [over 10 years ago](#).

§ 6.2. Separation of CPU and GPU

Fundamentally, both SFML and SDL have very clear concepts of where resources live (CPU vs. GPU), separating them into different types - e.g. the `sf::Image` type for CPU resources, and the `sf::Texture` type for GPU resources. `SDL_Surface` and `SDL_Texture` are analogous types in SDL.

A common workflow in these 2D libraries goes as such: Create an object that's stored on the CPU, load or create your asset, write it to the GPU once, then use that GPU object repeatedly. Users are free to modify data when it's stored on the CPU through conventional methods, and transfers to and from the GPU are generally obvious. While neither library provides the ability to do [proper](#) asynchronous or high performance GPU data transfers, the reality is that it is easy to distinguish as a user of the library whether an operation will be fast or slow, and performance is generally acceptable. There are no [hidden expensive costs](#) that might dramatically impact performance.

P0267 as currently stated runs into a double whammy of both performance, and functionality issues, mainly due to a lack of equivalent separation between CPU and GPU types.

§ 6.3. Performance Limitations

1. No definition of when necessary but undesirable behaviour like data transfers should occur, forcing implementations and users of the library to guess.
2. Using APIs performantly will vary between different implementations as a result, quite significantly, requiring code rewrites or even implementation `#ifdefs`.
3. Partially hardware accelerated backends may have extremely low performance behaviour that is very difficult to avoid as a result. Implementations may be forced to perform hidden readbacks that are very difficult to avoid.

Partial hardware acceleration is additionally a likely, and extremely undesirable outcome of P0267: partly due to its basis on Cairo's API, and partly due to the lack of specification or experience on how a realistic hardware accelerated implementation might look.

A library with correctly separated CPU and GPU types cannot reasonably suffer from these performance limitations, or partial hardware acceleration in particular, and is much more predictable to use.

§ 6.4. Functionality Limitations

1. Every proposed API must suffer from the union of worst case API requirements between hardware and software implementations.
2. Hardware and software implementations have very different API requirements.
3. Trying to fit these requirements together makes basic features hard to standardise, or requires complex APIs.

Some features like [direct pixel](#) modification/access for CPU types are standard and basic across a range of 2D graphics libraries. Direct pixel access however is not specified for GPU types, as it is significantly complex to specify and implement requiring a complex asynchronous API, and instead most libraries solely define [pixel transfer in bulk](#) between the CPU and GPU.

It is likely that direct pixel access of any form would be difficult to adequately specify in P0267, due to this problem. Batched rendering is an existing example of an overly complex API (discussed in detail under [§ 7 Performance, Safety, and API Usability](#)), and external surface modification and native interop were previously supported in this proposed 2D graphics library and both removed, potentially due to similar issues.

In the reference implementation, portable direct pixel access in the event of an unknown implementation is achieved by saving an image to disk, and then manually loading and decoding the resulting .png image through non 2D graphics APIs. Needless to say, this is not the most performant solution.

More generally, unifying all the different implementation constraints under one device agnostic API is challenging, and will likely result in an unnecessarily complex, unsafe, non performant, and hard to use APIs.

§ 7. Performance, Safety, and API Usability

§ 7.1. Fully Automatic Batching

Performance in a 2D graphics library is often played down, with it being deemed unnecessary that beginners might want to render large numbers of objects or create realtime interactive applications that run at >30fps, but there is reason to believe that the API as a whole might not be well implementable in the general case.

The key concept that P0267 relies on for good performance in hardware implementations, is that they fully batch, automatically. Batching is a big component of good performance once you stray out of

simple use cases, and the authors hope (as there is no batched reference implementation) that the API design will allow for it to be batched internally.

In theory this is a great idea. When looking at comparable libraries for this kind of generic graphics rendering though, like SFML and SDL, neither of them are fully automatically batched - despite being explicitly constructed such that internally they could be batched, similar to the 2D graphics proposal. Forks have been made of SFML which partially support this, and SDL partially batches internally.

The problem is that batching is not a singular concept. A member of the SFML team puts it like [this](#):

"I know what you are going to say now, why doesn't SFML do this for you since you think many might need this and this is what a multimedia library is for. I already mentioned the answer above. Different people will need different kinds of sprite batching. Writing an efficient sprite batcher is very application specific and having to account for every possible thing that developers might use it for would probably end up making it slower than simply drawing the sprites yourself. "

Skia is a notable example of a library that tries to automatically fully batch as much as possible and is very explicitly designed to take advantage of this. Even in the case of [Skia's batching](#), API limitations still result in batching being [difficult](#), for a similar reason in that automatic batching is difficult in the general case. APIs with limited and specific scope like Dear ImGui, a C++ UI toolkit, are able to fully automatically batch effectively. In the case of Dear ImGui, it does not expose a manual batching API as part of ["normal"](#) user code as it is unnecessary, but would suffer from similar difficulties in fully automatically batching if used as a general 2D graphics library.

Post hoc fixes to APIs like Cairo, Skia and WebRender to allow for more batching, or more efficient hardware acceleration, can result in large wide ranging architecture changes. For an example, [WebRender required](#) major overhauls in fixing what seems like a potentially non API design issue - much higher power consumption on macOS vs. Windows (even though both are fully hardware accelerated) due to a seemingly minor cross platform difference. Specifically, "These findings have informed substantial changes to WebRender's architecture". Given that C++ must be backwards compatible, it means that any very problematic issues discovered with the API design that may prevent acceptable performance and fully automatic batching in particular are likely permanent. This is less of a problem in APIs like SFML and SDL, as they tend to minimally wrap specific well understood existing graphics concepts, like a (specifically) GPU texture or a vertex array, instead of trying to specify behaviour in a higher level declarative sense and leaving the implementation very uncertain and complex to implement.

So while it is possible to batch automatically to some degree, it is not necessarily a full solution even if the API is well designed, for 2D graphics libraries. SFML and other more traditional graphics/media APIs instead solve this by providing an explicit API for submitting batched draw calls or vertex lists, which now brings us to the issue of safety.

§ 7.2. Performance Issues Lead To Unsafety

P0267 has tried to solve some of the probable or previously proposed performance issues with a high level batching API: it provides a `command_list` structure, which has a theoretically batching interface. The odd thing about this interface is that it is defined such that it can run on a separate thread returning a future, with undefined behaviour from data races that must be carefully managed by the user.

GPUs in OpenGL and DirectX implementations by and large do not benefit from multithreading. The primary purpose of this threading requirement appears to be to allow the implementation to support software renderers efficiently, as well as hide the cost of fully automatic batching - but introduces potential undefined behaviour through data races which must always be controlled by the user. On top of this, actually using OpenGL/etc. from a multithreaded perspective is tricky, as an OpenGL context can only be active in one thread at once, requiring additional synchronisation by the implementation to avoid incorrect behaviour.

Schemes like in this proposed API must do some kind of hidden processing on the data to determine which commands can be grouped together which can be expensive - simply submitting a group of high level commands at once does not make them batchable. This threading workaround potentially requires multiple mutexes (implementation and user facing), context switches (both thread switches, and backends like OpenGL), provides a future to tell you when the work has completed (must query state from the backend, this is not free!), all of which which can be very expensive, not to mention unsafe, and complex to get right for programmers who might use this graphics library. There isn't a compelling upside to this specification, but it is somewhat necessary due to the high level API design issue and the implicit batching goals of P0267.

Other batched rendering APIs do not suffer from these issues. SFML for example provides a straightforward batched rendering API equivalent (through vertex lists) that largely provides the performance benefits of batching with no unsafety.

This unsafety stems from a fairly fundamental issue with P0267. Due to the lack of distinction between CPU and GPU, any API specification must suffer from the union of the API requirements necessary to support either one of these devices correctly. The threading must ideally be present for software renderers to be maximally efficient, and the future is required due to this thread. A hardware renderer would not need this complex API (assuming it were more explicit) - but hardware rendering is what introduced the requirement of a (specifically) batching API in the first place which then must be supported efficiently in software renderers. This API cannot (and should not, in the current lack of separation model) be only targeted at only software renderers or hardware renderers, so we end up with this mix instead.

As-is, a single API needs to support too many use cases - explicit + implicit batching, mixed software rendering + hardware rendering, with hidden implicit threads - and it gives unnecessary beginner

unfriendly data races and poorer performance as the result. This is a sign of problems with the overall API design, as well as the goal of being fully automatically batched in the first place. This unsafety to get performance, however, leads to further problems.

§ 7.3. Unsafety Leads To Usability Limitations

As a result of the problems in the current API design, `command_list` is unsafe to use as it implicitly may create threads, versus an API like SFML which is implicitly safe. As a result of the unsafety of the batching system, the authors recommend that the graphics proposal be used like so:

"As such, it is recommended that users choose to use either command lists or the "direct" API (where surface member functions that perform rendering and composing operations, copying image surfaces, saving image data, etc.)."

In short, they recommend either using the batching interface only, or the normal interface only, and not mixing them. In a library like SFML, batched and non batched code can be easily, and safely mixed. This means that your program can be 'gradually' batched - only things that actually needs to be batched suffer the complexity from being explicitly batched, and everywhere else you can use the regular API. You can teach a beginner the relatively simple batched SFML API, introduce them to it piece by piece, and they can progressively rework their code to be batched only where it needs to be batched.

In the proposed API, a beginner needs to rework all their rendering code, or work with synchronisation primitives like mutexes and futures. This fundamentally is hard to teach as it mandates knowledge of threads, synchronisation, and async code, and presents an unnecessary hurdle for interested beginners trying to get better performance out of the library or learn about batching.

The sole reason that this is not easily possible in the graphics proposal is API design deficiencies. API design issues have led to performance issues, attempts to solve which have led to unsafety issues, which have led to API usability issues compared to the alternatives. There is a chain of serious issues that must be present for the API to have these problems.

§ 7.4. Potentially Unsolvable

As it stands with this batching API, there are several options, none of which are optimal

1. Keep it as it is. This has unnecessarily poor usability characteristics, and there are still performance issues.
2. Remove the implicit threads. This would potentially quite dramatically reduce performance on a software implementation.

3. Remove the batching API altogether. This would potentially quite dramatically reduce performance on a hardware implementation.
4. Add another function, and specify the two methods to be designed for hardware or software renderers specifically. This leads to two ways to do the same thing on one object, where the optimal API to use is dependent on the implementation which cannot be queried and is inherently unspecified.
5. Split up the types involved into CPU and GPU versions, and expose the minimal necessary APIs on each. This requires major changes to the 2D graphics proposal, but would solve many fundamental issues.

This kind of tradeoff is likely to be a common story for other future features (like direct pixel access), even though it is unnecessary. This I believe is a fundamental limiting factor in the kind of functionality that future versions of the 2D graphics library will be able to offer, as well as how usable and performant any functionality it can offer is.

§ 8. Bad For Beginners

Beginner teachability is often used as a justification for the current 2D graphics proposal, with the angle that *anything* is better than nothing. Whether or not this is true, there are several fundamental problems here that would cause me to actively discourage beginners from using the proposed 2D graphics library, if it were standardised today as-is (with some bugfixes applied).

1. Poor correctness - It provides no hand holding for colour management, and `rgba_color` is actively harmful. Every beginner is going to mess this up. There are numerous technical mistakes across the 2D graphics proposal and reference implementation, demonstrating the difficulties involved here.
2. Lack of good vocabulary types - The algebra types in general do not provide sufficient functionality for application development, which means writing a lot of code yourself or importing a second vector library. This is needlessly complex, and in some cases promotes writing incorrect code (colours).
3. Insufficient performance - The performance is likely limited by the API design. You might be able to create toy applications, but getting more out of it is a problem and may even result in hard to debug thread unsafe behaviour. Beginners would be trapped in a "beginners hole", unable to make video games (often the central goal of beginners to graphics) or moderately complex applications with the knowledge they've learnt. Fixing this is potentially all or nothing in user code, which hampers teachability compared to alternatives. Performance and power consumption on mobile or integrated devices is likely to be unacceptable in any situation on any backend.

4. Feature deficient by design - It mixes together concepts like CPU and GPU in a way that other libraries do not, making it hard to offer basic standardised functionality that a teacher would expect from being familiar with other graphics libraries like direct pixel access.
5. Underspecified behaviour - Getting correct behaviour is implementation defined. A beginner might not understand what's going on with premultiplied alpha or subpixel font rendering (both implementation defined), resulting in unfixable "my sprites don't blend well" or "this text is kind of blurry" problems that would require a different standard library to fix.
6. Hard to transfer skills - It does not provide a base teaching point for concepts that might be transferable to a more widespread graphics library, like SFML or SDL. It would largely be learning from scratch due to their very different APIs, whereas knowledge from SFML and SDL are relatively transferable to other APIs as they wrap ubiquitous graphics concepts.
7. Lack of real world experience - There is no existing code literature for games created in a similar API, like Cairo, whereas there are decades of answered beginner questions about APIs like SFML and SDL. The reference implementation is incorrect in major ways, and there is no fully automatically batched implementation available. It is not clear if a fully automatically batched renderer or even a fully hardware accelerated implementation with acceptable performance characteristics is possible, as there is no precedent in a Cairo style API for this. It is not clear if it is possible to build any games more complex than extremely simple in a Cairo style API. I could find none.
8. Probability of further unknown issues - Most features I have been able to review within my expertise have turned out to have multiple issues with them, some very concerning or basic, but there are large parts of the proposed 2D graphics library that I am incapable of assessing. Many issues that may be discovered after standardisation are unfixable. This is partly due to a fundamental problem with standardising unproven functionality into C++, and partly due to the kind of API being standardised.

§ 9. Conclusion and Recommendations

The linear algebra types provided by the proposed 2D graphics library are very inadequate generally, and in at least one case dangerous due to inadequate design. The graphics API fails to separate out the concepts of CPU and GPU, which leads to poor performance, unsafety, and a fundamental difficulty in providing basic features offered by other libraries. The text of P0267 and reference implementations both make extensive mistakes surrounding linear colour. As-is, the proposed 2D graphics library overall will be hampered in its future evolution without fundamental changes, and I do not believe it currently caters to any audience of developers, from extreme beginner to expert.

To fix this, the following changes need to happen:

1. P0267 needs to be split up into multiple, smaller, independent proposals. It is extremely difficult for experts to review it adequately due to its sheer size at 261 pages, and I suspect this is a major contributing factor to P0267's current state. Given that R6 was 148 pages long, P0267 is likely to keep increasing massively in size as a lot of functionality is still unspecified, like input, and it still only specifies basic drawing facilities. It would not surprise me to see P0267 hit at least 400-500 pages, whereas the working draft for the entire C++17 standard is [only 1448 pages](#) long.
2. A proper vector library needs to be made the basis of the linear algebra types, and proper colour theory needs to be done on a potential vector linear `rgba_color` type. The lack of a proper foundational library is likely a big reason why there are many mistakes and misunderstandings with `rgba_color`, colour spaces, and premultiplied alpha, in both the reference implementation and P0267 itself. Linear colour errors would have been impossible or significantly more obvious if based on a well designed library.
3. The graphics API needs to be changed from its current design to one where there is a clear distinction between CPU and GPU, with functionality appropriately specified between the two. These fields could potentially each be a separate, smaller proposal, and this separation allows the GPU side (the hardest to standardise part) of the graphics proposal to be shrunk significantly, with the majority of drawing functionality being specified on the CPU, constructing vertex data to be processed by the GPU.
4. Renderable objects like text and circles/etc. should be unified into the concept of a shape or renderable, which is instantiated and built (e.g. constructing vertices internally or explicitly), before being rendered or transferred to the GPU.
5. Fully automatic batching should be scrapped as a design goal. With #3 and #4 accomplished, it becomes easy to provide a batched API which is relatively straightforward to use, safe, higher performance, and can be gradually introduced in user code where necessary for performance instead of requiring changes to all user code. Automatic batching like SDL should be a design goal, but not with the intent that it is a full solution.
6. A correct reference implementation should be written and validated against a separate existing implementation, to validate the functionality of the proposed API, including performance. It is very difficult to conclusively prove theoretical performance properties (or lack thereof) about any graphics proposal as issues are often hard to see in advance from a high level API design, so performance arguments need to largely occur against real world code with minimal leaps of logic, across multiple platforms and vendors to root out issues. An API shouldn't be considered automatically batchable on a type of hardware until it has been explicitly shown to be, due to the often subtle complexities involved. It is not even clear or readily obvious that a full hardware implementation of P0267 without unacceptable hidden costs is possible, especially when the API it is based off explicitly states that it is not practical.

These necessary changes could be summed up more simply by saying that a Cairo-like API isn't well suited for hardware acceleration or realtime interactive applications, and SFML/SDL provide much more relevant functionality. A non toy videogame has not yet been written in Cairo, whereas both SFML and SDL are extensively used in a wide variety of applications and games. In the event that both above basic beginner videogames are deemed to not be in scope and fully automatic batching is desirable, Skia provides a good starting point additionally, potentially with inspiration from the cutting edge of web development technologies like WebRender, though declarative APIs overall likely present a much bigger standardisation problem.

Fixing P0267 as-is is equivalent to redoing the entire thing and starting again with a different base, with prerequisites on having a standardised linear algebra library (or at minimum a proper colour library), and deep review by relevant experts.

§ 10. Misc. Errors

This section is related to errors with P0267 that I found incidentally while writing this document, notable errors in the reference implementation, or are errors unrelated to the wider theme of the document.

§ 10.1. Linear Colour

- Premultiplication is defined as such:

"visual data format with one or more color channels and an alpha channel where each color channel is normalized and then multiplied by the normalized alpha channel value [Example: Given the 32-bit nonpremultiplied RGBA pixel with 8 bits per channel {255, 0, 0, 127} (half-transparent red), when normalized it would become {1.0f, 0.0f, 0.0f, 0.5f}. When premultiplied it would become {0.5f, 0.0f, 0.0f, 0.5f} as a result of multiplying each of the three color channels by the alpha channel value. - end example] "

Premultiplication should be done in linear space. Given that this is 8-bit rgba this is almost certainly sRGB, which means that the correct sequence of steps is: linearise, then premultiply, then optionally go back to sRGB if you wish to store your data in sRGB, or keep it as linear floats. Either way, this example only works by happenstance. Note: This is probably one of the most pervasive errors in computer graphics, it is very likely that at some point intermediate tools will handle either premultiplication incorrectly, or linear colour incorrectly. Even decoders for common formats like PNG can handle this incorrectly.

- The reference implementation implements incorrect linear colour handling in `ConvertToAlphaless` and premultiplied alpha in general, although it is dubiously 'correct' as

currently specified.

- The reference implementation provides functions to convert between HSV and `rgba_color` without linearisation. HSV is gamma encoded, and `rgba_color` is linear.
- The reference implementation appears to define the sRGB colour space `kCGColorSpaceGenericRGB` as the colour model being used, which is incorrectly used to process `rgba_color` which should be linear - though in reality incorrectly contains sRGB data. These two errors cancel out.
- The reference implementation as a whole is largely incorrect, and after more investigation does not handle any kind of sRGB or linear colour correctly. The test images were probably generated using the reference implementation itself, and nearly every generatable image which involves any kind of blending, interpolation, transparency, or colour of any kind is incorrect, quite significantly, across any backend that conforms to the test suite. Please see [this](#) demonstration for an explicit example demonstrating this.

§ 10.2. Others

- Text rendering in its current form allows you to specify render props, but the necessary blend mode specified when rendering text is dependent on whether or not you render grayscale or subpixel antialiased font - however currently text rendering modes are not mandatory of the implementation. Additionally, the blend mode needed to render subpixel antialiased fonts correctly is not present (a correct subpixel font rendering API requires dual source blending with the `GL_SRC1_COLOR` blend mode which is not universally standard), and it is not clear how this could be supported as the API does not support any kind of non standard functionality.
- The paper defines very basic interop with a window, but provides no functionality to actually make it usable. APIs like OpenGL are implemented as a global state machine, which means that the rendering behaviour of the implementation is dependent on global factors. Changing this state is expensive, and for this reason implementations seek to generally set up the global state and then change it as little as possible. When user code performs interop with the proposed 2D graphics library, changing any global state at all would be a hard piece of implementation defined behaviour - which means digging through source code, and severely hampers cross platform code. For this reason, SFML defines `pushGLStates`, `popGLStates`, and `resetGLStates`, as well as context functions, to standardise interop with its internal state. It isn't the most performant solution for very advanced user specified functionality, but it significantly eases the burden and is more than adequate for most beginner to advanced intermediate use cases of interop, which is already a moderately advanced use case. Something similar could and should be added that is generic across different backends. However, notably the threaded batch interface introduced in

P0267R9 interferes with interop due to OpenGL contexts only being able to be active on one thread at once, which results in even more complexity.

- The paper defines all composition operations to take place as-if in linear colour, which in some cases could require sRGB framebuffers which may be unavailable. It is not clear what should happen if at runtime the implementation detects it is not able to provide this guarantee.
- It is not clear if the fixed refresh rate mode takes into account the case where draw callbacks take longer than the time between frames. The wording looks like either it does not handle that case correctly, or that it neglects the time it takes for a draw callback itself to execute.
- The paper uses `std::function` for its callback types. While this isn't a dealbreaker or necessarily incorrect as such, it does seem unnecessarily heavy when comparable libraries do not suffer from this overhead. Low overhead C-style callback driven libraries (e.g. GLFW) tend to be trickier to use due to `void*` pointers (which P0267 also seems to contain, under `run_function`) or global state - whereas an event queue system like SFML or SDL maps well to the underlying model, provides minimal overhead, and is very safe. This requires a major redesign of the graphics proposal, but this paper is proposing a major redesign anyway.
- `constexpr rgba_color& operator*(U lhs, const rgba_color& rhs) noexcept;` should be `constexpr rgba_color operator*(U lhs, const rgba_color& rhs) noexcept;`, a similar very minor error is made with `constexpr rgba_color& operator*(const rgba_color& lhs, U rhs) noexcept;`.

§ 11. Footnotes

§ 11.1. Async

Due to lack of control over the memory space of where allocations happen in most higher level libraries, memory is generally allocated outside of PCI-E accessible memory, which necessitates that the driver must copy your memory elsewhere before it can be written to the GPU. (See: [OpenCL Memory Objects](#)).

Friendly APIs are generally blocking as well, because a true asynchronous API (even aside from the PCI-E accessible memory issue) either has to copy your memory buffer (for writes), or ask you not to use it for the duration of the transfer which is relatively complex (writes and reads). Neither of these issues are actually that bad for most code, except that synchronous reads largely produce unacceptably poor performance. A future change in API design however could include an allocator or allow users to specify storage types, as well as optional asynchronous operations, opening up the use of this library to high performance applications uncommonly associated with existing 2D graphics APIs, while still remaining beginner friendly for the common cases. [Back](#).

§ 11.2. Dear ImGui

Dear ImGui however does provide an interface that can be used for batching, by exposing its internal APIs. This is intended for custom or more advanced rendering, and is not necessary in code which just utilises ImGui's normal functionality. [Back](#).

§ 11.3. operator*

It is tempting to think that operator* could be used to convert between a 0 -> 1 format, and a 0 -> 255 format. P0267 specifies that operator* *saturates*, that is, rgba_color can never store a value > 1 in its components. For other transforms that one might do with operator* like multiplying by 0.9, it is incorrect to use on sRGB data if it was actually intended to store sRGB. That said, it breaks the API contract to multiply by 1.1 (or any value > 1), so it is unclear overall what the purpose of this operator is. [Back](#).

§ 11.4. Hidden API Cost

While a library like SFML generally lacks large hidden performance costs, that does not also mean that the APIs are *fast*. The main concern is allowing users to avoid APIs that are [inherently and obviously slow](#) (a GPU read) or intentionally signpost their usage, rather than accidentally uncovering such behaviour as an implementation defect or defect in the design of the proposed 2D graphics API. [Back](#).